

Tanawu3 - Arabic Dialect Identification

Emery Jacobowitz
Brandeis University

1 Introduction

Language identification (LID) is a longstanding task for NLP researchers. There are numerous reasons for which it may be useful to identify the language of arbitrary text, such as the delegation of language-specific tools and lexical resources.

Many, if not most, languages have multiple spoken varieties, and differentiating between them can be just as useful. The Arabic language is spoken by over 400 million people, in a geographic area spanning from Morocco to Iran, with the addition of numerous enclaves and diasporic groups beyond this region. The standard variety, *Modern Standard Arabic* (MSA), is used as a common written language, and is used in formal and educational settings. Yet for all other domains, Arabic speakers use regional dialects.

The Arab world can be viewed as a large dialect continuum, where regional varieties may differ on many linguistic levels, including phonology, morphology, and syntax. There is no consensus on the precise meaning of *dialect* in this context, as some scholars describe the Arabic dialects as separate languages, while others treat them more traditionally as varieties of a single language. The status of mutual intelligibility is also unclear, especially in the era of mass media where speakers can be exposed to many different dialects. Most dialects are assumed to be mutually intelligible, though this may not be true for speakers on distant frontiers of the continuum.¹

1.1 Task

The goal of this project is to predict with high accuracy the dialect of arbitrary samples of Arabic text. This is a multiclass classification problem, as each text can be classified as belonging to one of

five dialects:

- 1) Egyptian (EGY)
- 2) Levantine (LEV)
- 3) Iraqi (IRQ)
- 4) Gulf (GLF)
- 5) Maghrebi (MGH)

Note that these dialects are themselves abstract groups of similar dialects. LEV, for example, includes both North (Syrian-Lebanese) and South (Palestinian) Levantine Arabic.

2 Data

The dataset used in this project is the **Dialectical ARabic Tweets (DART)** corpus compiled by (Al-sarsour et al., 2018). The corpus contains approximately 25,000 Arabic-language tweets, excluding tweets deemed too short, duplicated, or inappropriate by the compilers. Tweets were first programmatically pseudo-labeled by word lists, then labeled by crowd-sourced annotators located in the appropriate dialect regions. The crowd-sourced annotations were filtered to include only labels where a majority of annotators agreed. Finally, labels were reviewed by native speakers.

The breakdown of labels in the corpus is shown in Table 1. While there is some disparity in the label frequencies, overall they are relatively similar.

Several postprocessing were performed on the DART data. Non-linguistic features such as links, numbers, and emoji were removed. Arabic characters were regularized, removing vowel diacritics and character variations. Before classification, tweets were tokenized by splitting on word boundaries.

¹I'm not citing any sources here because I didn't have time or energy to put together a complete bibliography. Please do not take my word for any of these claims.

Label	Count	Freq.
GLF	5,893	24%
EGY	5,265	22%
IRQ	5,253	22%
LEV	3,939	16%
MGH	3,930	16%
Total	24,280	100%

Table 1: Label frequency in the DART corpus.

3 Dev Set Results

The two classification models used for this task were **Multinomial Naive Bayes** and **Logistic Regression**. Each model was evaluated with several different hyperparameter values.

3.1 Data Splits

The corpus was pseudorandomly split into train, dev, and test subsets. Train contained 70% of the tweets, dev contained 10%, and test contained 30%. The test set was reserved solely for final model evaluation.

3.2 Feature Sets

In addition to model hyperparameters, this approach involved a significant amount of feature engineering. Three different approaches to feature extraction were attempted.

3.2.1 Bag of Words (BOW)

The simplest features sets are unigram and bigram BOW, where each feature corresponds to the presence of a token (or token bigram). Binary feature values were used instead of counts due to the short length of most tweets.

3.2.2 Stop Words

Not all tokens are equally valuable. Tokens appearing frequently across all classes are termed *stop words*. Note that this differs from the traditional usage of the term, which refers mostly to common functional-class words which carry little lexical information. Stop words were determined by sorting train tokens by frequency within each label. Tokens were then sorted again by average rank across all labels. The n most common labels were discarded, where n was a configurable hyperparameter, with values of 0, 50, and 100 tested.

3.2.3 Embeddings

To capture the semantic, as opposed to simply lexical, content of each tweet, two embedding models

were used with logistic regression. Each model was trained on the train split. **word2vec** embeddings of each token were averaged across the tweet in each dimension. **doc2vec** embeddings captured the entire tweet. Each embedding scheme was tested with vectors of size 30, 50, and 100. Both models were trained for 10 epochs due to timing limitations²

3.3 Hyperparameter Search

An extensive grid search was conducted on the dev set to determine optimal hyperparameters based on accuracy. Each configuration included a specific feature set (SW refers to stop words) and the model hyperparameter. All logistic regression models were trained with 1000 maximum iterations. The results of the logistic regression search are shown in Table 2.

The most successful features were unigram BOW. The highest-accuracy configuration was unigram BOW with 100 stop words and $\alpha = 10$, at 95.86%. Other unigram feature sets also came close, and there does not seem to be a marked difference between 50 and 100 stop words, though the removal of stop words does seem to boost accuracy by a minor amount. The embedding models all performed poorly, likely to due in part to constraints on training epochs. doc2vec takes an especially long time to train, so it was only run with three configurations. word2vec also demonstrated much higher sensitivity to C than any of the non-embedded feature sets.

The naive Bayes dev set results are displayed in Table 3. Accuracy appears to be lower, with the best configuration of unigram BOW with 100 stop words and $\alpha = 5.0$ at 93.19%.

4 Test Set Results

The top performing models of each type, as well as one logistic regression model with word2vec features, were selected to be evaluated on the test set. The results of this evaluation are shown in Table 4.

For the best configuration, a more detailed statistical analysis is shown in Table 5.

A more detailed confusion matrix for this model is shown in Figure 1.

²That is, they were run very late at night.

Features	C	Accuracy
Unigram BOW	0.01	89.66
Unigram BOW	0.1	93.84
Unigram BOW	1.0	95.39
Unigram BOW	10.0	95.34
Unigram BOW	100.0	95.17
Unigram BOW	inf	95.30
Bigram BOW	0.01	54.66
Bigram BOW	0.1	67.59
Bigram BOW	1.0	71.25
Bigram BOW	10.0	71.77
Bigram BOW	100.0	71.94
Bigram BOW	inf	72.03
Unigram - 50 SW	0.01	89.09
Unigram - 50 SW	0.1	94.31
Unigram - 50 SW	1.0	95.60
Unigram - 50 SW	10.0	95.78
Unigram - 50 SW	100.0	95.30
Unigram - 50 SW	inf	95.22
Unigram - 100 SW	0.01	89.22
Unigram - 100 SW	0.1	94.22
Unigram - 100 SW	1.0	95.69
Unigram - 100 SW	10.0	95.86
Unigram - 100 SW	100.0	95.52
Unigram - 100 SW	inf	95.47
word2vec - 30	0.01	68.88
word2vec - 30	0.1	73.75
word2vec - 30	1.0	75.91
word2vec - 30	10.0	77.24
word2vec - 30	100.0	77.03
word2vec - 30	inf	77.72
word2vec - 50	0.01	68.53
word2vec - 50	0.1	73.66
word2vec - 50	1.0	78.66
word2vec - 50	10.0	79.31
word2vec - 50	100.0	81.08
word2vec - 50	inf	80.52
word2vec - 100	0.01	65.95
word2vec - 100	0.1	73.19
word2vec - 100	1.0	77.89
word2vec - 100	10.0	81.64
word2vec - 100	100.0	84.05
word2vec - 100	inf	83.41
doc2vec - 30	10.0	71.59
doc2vec - 50	10.0	71.80
doc2vec - 100	10.0	70.42

Table 2: Dev Set Accuracy - Logistic Regression

Features	α	Accuracy
Unigram BOW	0.001	80.91
Unigram BOW	0.01	84.40
Unigram BOW	0.1	89.87
Unigram BOW	0.5	92.16
Unigram BOW	1.0	92.24
Unigram BOW	2.0	91.81
Unigram BOW	5.0	90.39
Bigram BOW	0.001	74.44
Bigram BOW	0.01	74.91
Bigram BOW	0.1	75.56
Bigram BOW	0.5	75.39
Bigram BOW	1.0	74.91
Bigram BOW	2.0	74.27
Bigram BOW	5.0	71.68
Unigram - 50 SW	0.001	80.43
Unigram - 50 SW	0.01	84.14
Unigram - 50 SW	0.1	89.27
Unigram - 50 SW	0.5	92.11
Unigram - 50 SW	1.0	92.59
Unigram - 50 SW	2.0	92.80
Unigram - 50 SW	5.0	92.41
Unigram - 100 SW	0.001	80.13
Unigram - 100 SW	0.01	83.84
Unigram - 100 SW	0.1	89.09
Unigram - 100 SW	0.5	92.76
Unigram - 100 SW	1.0	92.93
Unigram - 100 SW	2.0	92.97
Unigram - 100 SW	5.0	93.19

Table 3: Dev Set Accuracy - Naive Bayes

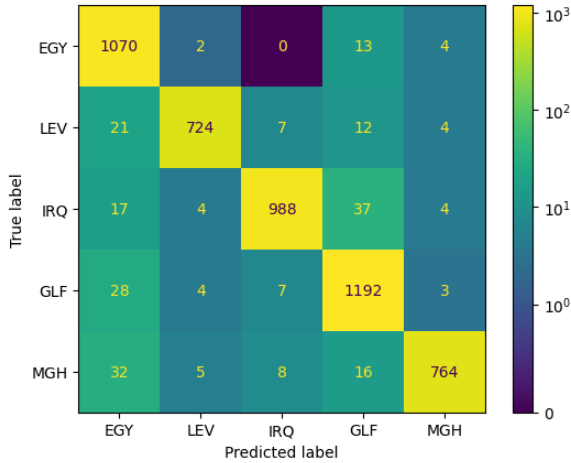
Model	Features	α/C	Accuracy
LR	Unigram BOW - 100 SW	10.0	95.41
LR	word2vec - 100	100.0	77.61
NB	Unigram BOW - 100 SW	5.0	93.03

Table 4: Test Set Accuracy

Class	Precision	Recall	F1-Score	Support
EGY	91.61	98.26	94.82	1089
LEV	97.97	94.27	96.08	768
IRQ	97.82	94.10	95.92	1050
GLF	93.86	96.60	95.21	1234
MGH	98.07	92.61	95.26	825
macro avg	95.87	95.16	95.46	4966
weighted avg	95.54	95.41	95.42	4966
accuracy	95.41			

Table 5: Test Set Classification Report

Figure 1: Test Set Confusion Matrix



5 Discussion

5.1 Feature Selection

It is unsurprising that unigrams outperformed bigrams. The bigram feature space is much larger than the unigram space, and due to the short length of most tweets, such feature vectors tend to be incredibly sparse. Simple unigram BOW features are sufficient to capture various differences in lexical items, which is one of the most obvious ways in which dialects vary.

This is also a likely factor in the unsatisfactory performance of embeddings as feature sets. While the low accuracy of doc2vec suggests that it did not suit the nature of the tweets, word2vec embeddings performed better, though still much worse than unigram BOW. This suggests that the best way to differentiate dialects does not involve the semantic function of each token, but rather its lexical properties. Some tokens are used at vastly different frequencies in different dialects.

Furthermore, it is possible that the same token could have different semantic properties in different dialects. However, non-contextual embeddings will not capture these different senses. A contextual embedding model like BERT could serve as a more powerful feature extractor.

The removal of stop words was not sufficient to improve a poorly-performing feature set. However, it was able to marginally improve the accuracy when the model was otherwise performant, by removing redundant features.

5.2 Error Analysis

The high accuracy results for this task are very promising. This project has demonstrated that di-

alect identification is a complex yet feasible task worthy of scholarly attention.

Yet this is far from a solved problem. In Table 5 and Figure 1, it is noticeable that while recall was relatively constant across labels, the model tended to exhibit lower precision for EGY and GLF. At first glance this appears to be the result of a frequency effect, as these labels were two of the most common. However, IRQ is similarly frequent to EGY, so this is not a simple answer. Another contributing factor could involve the quality of the data.

5.3 Data Limitations

While the DART corpus is a high-quality dataset, it is still beholden to some inherent limitations. The most important is that it only contains tweets which *can with relative ease* be labeled as belonging to a certain dialect. There are rich dialectal differences, such as the entire realm of phonology, which are lost in text. There are other variations, such as those involving syntax, which may be associated with certain dialects, but are not salient to untrained annotators. Because of this, any model trained on such a corpus is unlikely to perform well on real-world unannotated data.

Furthermore, not all dialects are represented equally across Arabic-speaking societies. Egyptian is the most prevalent dialect in media, while Gulf Arabic is sometimes considered the closest to MSA and is increasingly used in the domain of international business.³ Different speakers may have different associations and levels of exposure to different dialects. While the corpus attempts to control for this, mislabelings are ultimately unavoidable.

6 Conclusion

A logistic regression model with a unigram BOW feature set and stop word removal was able to achieve sufficiently high accuracy on the Arabic dialect identification task. Much of this researcher's effort was focused on feature engineering. While the optimal features only achieved minor improvements over the baseline, the baseline itself was quite accurate. However, it should neither be concluded that the task is solved, nor that any future improvements will be marginal at best. There are still many avenues for future work. It would be interesting to use different feature sets, such as the

³Again, this is an uncited combination of my own reading and anecdote from acquaintances.

aforementioned contextual embeddings, or smarter approaches capable of capturing morphological and syntactic structure.

References

Israa Alsarsour, Esraa Mohamed, Reem Suwaileh, and Tamer Elsayed. 2018. Dart: A large dataset of dialectal arabic tweets. In *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC'18)*, pages 3666–3670.